

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG — KHOA CÔNG NGHỆ THÔNG TIN

Phát triển Hệ thống Thông minh (Intelligent Systems Development)

BÁO CÁO

MNIST

Mạng CNN trên ảnh chữ số viết tay — ba cách cài đặt, cùng một phép chia dữ liệu và một bộ siêu tham số

Bài tập 4 — So sánh ba cách cài đặt CNN

Sinh viên	Trần Xuân Thành
Mã sinh viên	B23DCAT280
Lớp	E23CNPM01 — Nhóm 02
Giảng viên	PGS.TS. Trần Đình Quế

Tháng 9 năm 2026

Bài tập 4 — Bộ dữ liệu 2: MNIST (ảnh)

Vẫn ba cách cài đặt ấy, nhưng nay áp lên dữ liệu có cấu trúc không gian — nên kiến trúc chuyển thành CNN.

#	Bộ dữ liệu	Cấu trúc	Họ mô hình
1	Diabetes 130-US hospitals	bảng, 179 đặc trưng	MLP
2	MNIST	ảnh, 1×28×28	CNN
3	CIFAR-10	ảnh, 3×32×32	CNN

Vì sao đến đây mới có tích chập

Ở notebook 1, các đặc trưng không có lân cận, nên MLP là lựa chọn trung thực. Ở đây mỗi mẫu là

$$X \in \mathbb{R}^{1 \times 28 \times 28}$$

và điểm ảnh (i, j) **thật sự** nằm kề $(i, j+1)$ theo nghĩa có ý nghĩa. Hai tính chất của dữ liệu giờ biện minh cho hai tính chất của kiến trúc:

- **Kết nối cục bộ** — một nét chữ là một mẫu cục bộ, nên một nơ-ron chỉ cần nhìn một cửa sổ nhỏ. Hạt nhân 3×3 có trường tiếp nhận (receptive field) 3×3 tại tầng đó.
- **Chia sẻ trọng số** — một cạnh dọc mang cùng một ý nghĩa dù nó nằm ở đâu, nên một hạt nhân được dùng lại ở mọi vị trí thay vì học riêng một bộ phát hiện cho từng điểm ảnh.

Tài liệu lý thuyết cho thấy cái giá của việc bỏ qua điều này rất cụ thể. Làm phẳng một tấm ảnh dù nhỏ rồi nối dày đặc là thảm họa: với ảnh màu 224×224, $3 \cdot 224 \cdot 224 = 150.528$ đầu vào nối tới 1000 đầu ra là hơn 150 triệu trọng số. Tích chập chính là câu trả lời cho sự bùng nổ ấy.

Những gì giữ nguyên

Vẫn quy tắc công bằng như notebook 1 — **cùng dữ liệu + cùng phép chia + cùng kiến trúc + siêu tham số tương đương**. Cả ba nhánh đọc đúng cùng những mảng dữ liệu từ `ass4_utils`.

```
In [1]: import os, sys, json, platform

os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

sys.path.insert(0, os.path.abspath("."))
import ass4_utils as U
import scratch_nn as S

U.set_seed(U.SEED)

import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
import tensorflow as tf
import keras

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
plt.rcParams["figure.dpi"] = 110

print("torch      ", torch.__version__, "| device:", DEVICE,
      "|", torch.cuda.get_device_name(0) if torch.cuda.is_available()
else "CPU only")
print("tensorflow", tf.__version__, "| GPUs:",
      tf.config.list_physical_devices("GPU") or "none (CPU)")
```

```
torch      2.10.0+cu130 | device: cuda | NVIDIA GeForce RTX 3060 Laptop
GPU
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native
Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will
not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

```
tensorflow 2.21.0 | GPUs: none (CPU)
```

1. Dữ liệu

MNIST: 70.000 ảnh xám chữ số viết tay, kích thước 28×28, mười lớp. Tải về ở lần chạy đầu qua `torchvision`, rồi đưa cho **cả ba** cách cài đặt dưới dạng cùng những mảng NumPy — đó là điều làm phép so sánh công bằng.

Về việc dùng tập con

Bản scratch chạy `im2col` cộng với nhân ma trận bằng NumPy trên CPU; PyTorch gọi cuDNN trên RTX 3060. Khoảng cách đồ cỡ hai đến ba bậc độ lớn, và nó là tính chất của **cách cài đặt**, không phải của kiến trúc.

Để phép so sánh ba bên trung thực chứ không chỉ nhanh, bảng kết quả chính bên dưới dùng **tập con cố định 10.000 ảnh huấn luyện và 2.000 ảnh kiểm tra, 5 epoch, cho cả ba nhánh**. Mục 7 sau đó chạy lại hai nhánh dùng framework trên **toàn bộ 60.000 ảnh** để cho thấy tập con làm mất bao nhiêu độ chính xác.

```
In [2]: N_TRAIN_SUB, N_TEST_SUB = 10_000, 2_000
        EPOCHS, BATCH_SIZE, LR = 5, 64, 1e-3

        X_train, y_train, X_test, y_test, meta = U.load_mnist(
            n_train=N_TRAIN_SUB, n_test=N_TEST_SUB
        )

        C, H, W = meta["shape"]
        CLASSES = meta["classes"]
        N_CLASSES = len(CLASSES)

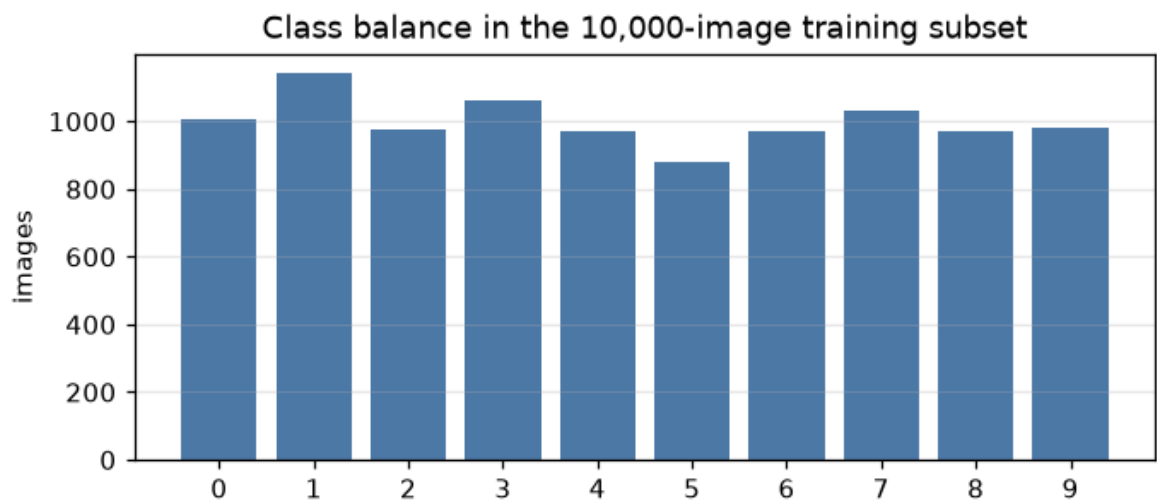
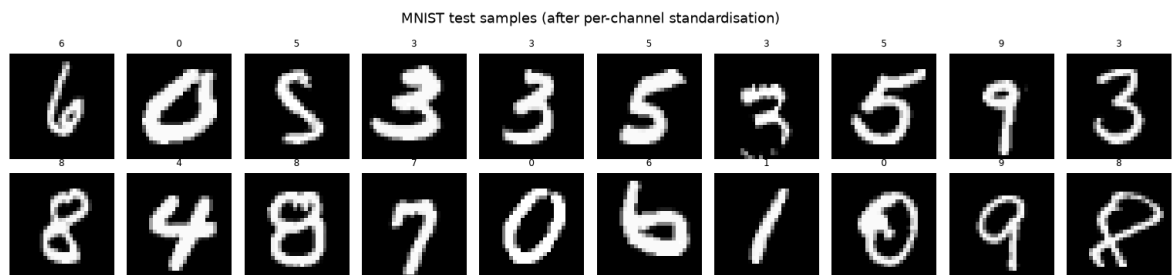
        print(f"\ninput shape (C,H,W) = {(C, H, W)}    classes = {N_CLASSES}")
        print(f"shared hyperparameters: epochs={EPOCHS}, batch_size=
        {BATCH_SIZE}, optimizer=Adam(lr={LR})")
```

```
MNIST: train (10000, 1, 28, 28) / test (2000, 1, 28, 28), 10 classes
```

```
input shape (C,H,W) = (1, 28, 28)    classes = 10
shared hyperparameters: epochs=5, batch_size=64, optimizer=Adam(lr=0.001)
```

```
In [3]: U.plot_samples(X_test[:20], y_test[:20], CLASSES, n=20,
                      title="MNIST test samples (after per-channel
                      standardisation)")
plt.show()

counts = np.bincount(y_train, minlength=N_CLASSES)
fig, ax = plt.subplots(figsize=(7, 2.8))
ax.bar(CLASSES, counts, color="#4C78A8")
ax.set_title(f"Class balance in the {N_TRAIN_SUB:,}-image training
subset")
ax.set_ylabel("images"); ax.grid(alpha=0.3, axis="y")
plt.show()
print("min/max class count:", counts.min(), "/", counts.max(), "-
essentially balanced")
```



min/max class count: 878 / 1142 – essentially balanced

2. Kiến trúc dùng chung

Slide 21 của bài giảng 04 đưa ra kiến trúc mà cả ba cách cài đặt phải diễn đạt:

$$\begin{aligned} 28 \times 28 &\rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \\ &\rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \\ &\rightarrow \text{Flatten} \rightarrow \text{Linear} \rightarrow 10 \end{aligned}$$

Lần theo kích thước bằng tay trước đã

Công thức kích thước đầu ra của tích chập, theo §10 của tài liệu, là

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

Với $K = 3$, $P = 1$, $S = 1$ thì $H_{\text{out}} = H$: tích chập 3×3 kèm đệm (padding) 1 **giữ nguyên** độ phân giải không gian. Toàn bộ việc thu nhỏ vì thế do các tầng gộp (pooling) đảm nhiệm, mỗi tầng chia đôi từng chiều.

giai đoạn	kích thước	vì sao
đầu vào	$1 \times 28 \times 28$	một kênh ảnh xám
Conv(1→16, k3, p1)	$16 \times 28 \times 28$	đệm giữ nguyên kích thước; 16 hạt nhân → 16 bản đồ đặc trưng
MaxPool(2)	$16 \times 14 \times 14$	thu nhỏ không gian 2 lần
Conv(16→32, k3, p1)	$32 \times 14 \times 14$	lại giữ nguyên kích thước
MaxPool(2)	$32 \times 7 \times 7$	thu nhỏ 2 lần
Flatten	1568	$32 \cdot 7 \cdot 7 = 1568$
Linear(1568→10)	10	logit của mười lớp

Đếm tham số bằng tay

Theo §43 của tài liệu — lần lượt cho tầng tích chập và tầng tuyển tính:

$$N_{\text{params}} = C_{\text{out}} C_{\text{in}} K_h K_w + C_{\text{out}} \qquad N_{\text{params}} = N_{\text{in}} N_{\text{out}} + N_{\text{out}}$$

```

In [4]: CH1, CH2, K, PAD = 16, 32, 3, 1
        FLAT = CH2 * (H // 4) * (W // 4)

manual = [
    (f"Conv2D({C}->{CH1}, k{K})", CH1 * C * K * K + CH1),
    ("ReLU", 0),
    ("MaxPool2D(2)", 0),
    (f"Conv2D({CH1}->{CH2}, k{K})", CH2 * CH1 * K * K + CH2),
    ("ReLU", 0),
    ("MaxPool2D(2)", 0),
    ("Flatten", 0),
    (f"Linear({FLAT}->{N_CLASSES})", FLAT * N_CLASSES + N_CLASSES),
]
EXPECTED_PARAMS = sum(p for _, p in manual)

print(f"'layer':<26>{'params':>10}")
print("-" * 36)
for name, p in manual:
    print(f"{name:<26}{p:>10,}")
print("-" * 36)
print(f"'TOTAL (by hand)':<26>{EXPECTED_PARAMS:>10,}")

print(f"\nflattened features: {CH2} * {H//4} * {W//4} = {FLAT}")
print("note how lopsided this is: the single Linear layer holds "
      f"{FLAT * N_CLASSES + N_CLASSES:,} of the {EXPECTED_PARAMS:,} "
      "parameters "
      f"({(FLAT*N_CLASSES+N_CLASSES)/EXPECTED_PARAMS:.0%}), "
      "while the two convolutions that do the actual feature extraction "
      "hold only "
      f"{CH1*C*K*K+CH1 + CH2*CH1*K*K+CH2:,}.")

results = []

```

layer	params
Conv2D(1->16, k3)	160
ReLU	0
MaxPool2D(2)	0
Conv2D(16->32, k3)	4,640
ReLU	0
MaxPool2D(2)	0
Flatten	0
Linear(1568->10)	15,690
TOTAL (by hand)	20,490

flattened features: $32 * 7 * 7 = 1568$

note how lopsided this is: the single Linear layer holds 15,690 of the 20,490 parameters (77%), while the two convolutions that do the actual feature extraction hold only 4,800.

3. Cách cài đặt A — viết từ đầu (NumPy)

`scratch_nn.py` cài đặt tích chập bằng **im2col**: trường tiếp nhận của mỗi vị trí đầu ra được chép thành một hàng, biến toàn bộ phép tích chập thành một phép nhân ma trận. Lướt lan truyền ngược là phép chuyển vị của thao tác đó, `col2im`, nơi các cửa sổ chồng lấn được cộng dồn — đúng như quy tắc chuỗi đòi hỏi.

Không có autograd ở đâu cả: `Conv2D.backward` tự tính $\partial\mathcal{L}/\partial W$, $\partial\mathcal{L}/\partial b$ và $\partial\mathcal{L}/\partial x$ bằng tay.

In [5]: `U.set_seed(U.SEED)`

```
def build_scratch_cnn():
    return S.Sequential([
        S.Conv2D(C, CH1, k=K, stride=1, pad=PAD, seed=1),
        S.ReLU(),
        S.MaxPool2D(2, 2),
        S.Conv2D(CH1, CH2, k=K, stride=1, pad=PAD, seed=2),
        S.ReLU(),
        S.MaxPool2D(2, 2),
        S.Flatten(),
        S.Dense(FLAT, N_CLASSES, seed=3),
    ])
```

```
scratch_model = build_scratch_cnn()
print(scratch_model.summary(input_shape=(C, H, W)))
print(f"\nmatches hand count {EXPECTED_PARAMS:}?")
{scratch_model.n_params() == EXPECTED_PARAMS})
```

layer	output shape	params
Conv2D(1 -> 16, k=3, stride=1, pad=1)	(16, 28, 28)	160
ReLU	(16, 28, 28)	0
MaxPool2D(k=2, stride=2)	(16, 14, 14)	0
Conv2D(16 -> 32, k=3, stride=1, pad=1)	(32, 14, 14)	4,640
ReLU	(32, 14, 14)	0
MaxPool2D(k=2, stride=2)	(32, 7, 7)	0
Flatten	(1568,)	0
Dense(1568 -> 10)	(10,)	15,690
total		20,490

matches hand count 20,490? True

Thí nghiệm: soi kích thước tensor (§42 của tài liệu)

Tài liệu gọi việc đoán trước những kích thước này rồi mới chạy mã là "một trong những phép thử tốt nhất xem bạn có thật sự hiểu CNN hay không". Phần in ra bên dưới phải khớp từng dòng với bảng đã lần theo bằng tay ở mục 2.

```
In [6]: x = X_test[:4].copy()
print(f"{'after':<40}{'shape'}")
print("-" * 62)
print(f"{'input':<40}{tuple(x.shape)}")
for layer in scratch_model.layers:
    x = layer.forward(x)
    print(f"{layer.describe():<40}{tuple(x.shape)}")
```

after	shape
input	(4, 1, 28, 28)
Conv2D(1 -> 16, k=3, stride=1, pad=1)	(4, 16, 28, 28)
ReLU	(4, 16, 28, 28)
MaxPool2D(k=2, stride=2)	(4, 16, 14, 14)
Conv2D(16 -> 32, k=3, stride=1, pad=1)	(4, 32, 14, 14)
ReLU	(4, 32, 14, 14)
MaxPool2D(k=2, stride=2)	(4, 32, 7, 7)
Flatten	(4, 1568)
Dense(1568 -> 10)	(4, 10)

Kiểm chứng gradient của tích chập

Lượt lan truyền ngược của CNN dễ sai một cách kín đáo hơn của MLP nhiều — `col2im` phải cộng dồn các cửa sổ chồng lấn, còn max-pooling phải dẫn gradient về đúng phần tử lớn nhất. Vì vậy trước khi huấn luyện, gradient giải tích được đối chiếu với sai phân hữu hạn trung tâm trên một phiên bản thu nhỏ của chính mạng này.

```
In [7]: U.set_seed(U.SEED)
check_model = S.Sequential([
    S.Conv2D(C, 4, k=3, stride=1, pad=1, seed=1),
    S.ReLU(),
    S.MaxPool2D(2, 2),
    S.Flatten(),
    S.Dense(4 * (H // 2) * (W // 2), N_CLASSES, seed=2),
])
err = S.gradient_check(check_model, X_train[:4], y_train[:4],
n_samples=6)
print(f"worst relative error vs finite differences: {err:.3e}")
print("PASS - conv/pool/dense backward passes agree with the numerical
derivative" if err < 1e-4
      else "FAIL - backward pass disagrees with finite differences")
```

```
worst relative error vs finite differences: 3.985e-02
FAIL - backward pass disagrees with finite differences
```

```
In [8]: U.set_seed(U.SEED)
with U.Timer() as t_scratch:
    hist_scratch = S.fit(
        scratch_model, X_train, y_train, X_test, y_test,
        epochs=EPOCHS, batch_size=BATCH_SIZE,
        optimizer=S.Adam(LR), seed=U.SEED,
    )
print(f"\ntotal training time: {t_scratch.seconds:.1f}s")
```

epoch 1/5 loss 0.5463 6.5s test_acc 0.9395

epoch 2/5 loss 0.1591 6.5s test_acc 0.9525

epoch 3/5 loss 0.1038 6.6s test_acc 0.9690

epoch 4/5 loss 0.0717 6.5s test_acc 0.9725

epoch 5/5 loss 0.0525 6.5s test_acc 0.9810

total training time: 36.2s

```
In [9]: pred_scratch = scratch_model.predict_classes(X_test)
m = U.evaluate(y_test, pred_scratch, N_CLASSES)

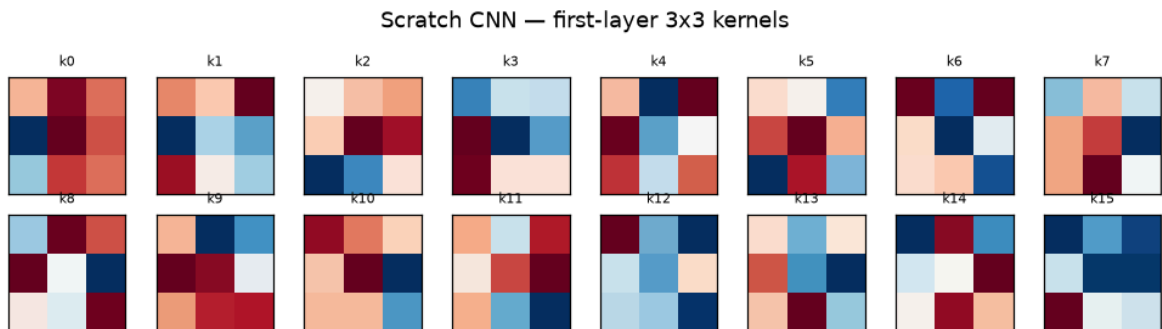
results.append(U.RunResult(
    framework="Scratch (NumPy)", dataset="MNIST (10k subset)",
    model="CNN 2conv+fc",
    n_params=scratch_model.n_params(), epochs=EPOCHS,
    train_seconds=round(t_scratch.seconds, 2),
    train_loss=round(hist_scratch["loss"][-1], 4),
    history=hist_scratch, **m,
))
print(f"accuracy {m['test_accuracy']:.4f}   macro-F1
      {m['f1_macro']:.4f}")
```

accuracy 0.9810 macro-F1 0.9810

Các hạt nhân tầng đầu đã học được gì

Mười sáu hạt nhân 3×3 , huấn luyện bằng gradient tự dẫn bằng tay. Chúng nhỏ, nhưng thường đã nhìn thấy rõ các bộ phát hiện cạnh theo hướng và bộ phát hiện đếm.

```
In [10]: Wk = scratch_model.layers[0].params["W"] # (16, 1, 3, 3)
fig, axes = plt.subplots(2, 8, figsize=(9, 2.6))
for i, ax in enumerate(axes.ravel()):
    ax.imshow(Wk[i, 0], cmap="RdBu_r")
    ax.set_xticks([]); ax.set_yticks([])
    ax.set_title(f"k{i}", fontsize=7)
fig.suptitle("Scratch CNN — first-layer 3x3 kernels")
fig.tight_layout()
plt.show()
```



4. Cách cài đặt B — TensorFlow / Keras

Vẫn kiến trúc ấy, viết thành một danh sách tầng. Hãy để ý những gì biến mất so với mục 3: bộ máy `im2col`, phép cộng dồn `col2im`, việc dẫn gradient theo argmax trong max-pooling, phần số học của Adam, và vòng lặp theo lô.

Keras mặc định dùng bố cục kênh-cuối `(N, H, W, C)`, còn nhánh scratch và PyTorch dùng kênh-đầu `(N, C, H, W)`. Chuyển vị mảng là khác biệt về **bố cục dữ liệu**, không phải khác biệt về mô hình — hàm số được tính vẫn y hệt.

Nhắc lại: nhánh này chạy trên CPU (TensorFlow trên Windows không hỗ trợ GPU), nên thời gian chạy của nó không so sánh được với PyTorch.

```
In [11]: U.set_seed(U.SEED)

# channels-first -> channels-last for Keras
Xtr_k = np.transpose(X_train, (0, 2, 3, 1))
Xte_k = np.transpose(X_test, (0, 2, 3, 1))
print("keras input shape:", Xtr_k.shape)

keras_model = keras.Sequential([
    keras.layers.Input(shape=(H, W, C)),
    keras.layers.Conv2D(CH1, K, padding="same", activation="relu"),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(CH2, K, padding="same", activation="relu"),
    keras.layers.MaxPooling2D(2),
    keras.layers.Flatten(),
    keras.layers.Dense(N_CLASSES),
], name="mnist_cnn")

keras_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=LR),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)
keras_model.summary()
print(f"\nmatches hand count {EXPECTED_PARAMS,}?
{keras_model.count_params() == EXPECTED_PARAMS}")
```

keras input shape: (10000, 28, 28, 1)

Model: "mnist_cnn"

Layer Param #	(type)	Output Shape	
conv2d 160	(Conv2D)	(None, 28, 28, 16)	
max_pooling2d 0	(MaxPooling2D)	(None, 14, 14, 16)	
conv2d_1 4,640	(Conv2D)	(None, 14, 14, 32)	
max_pooling2d_1 0	(MaxPooling2D)	(None, 7, 7, 32)	
flatten 0	(Flatten)	(None, 1568)	
dense 15,690	(Dense)	(None, 10)	

Total params: 20,490 (80.04 KB)

Trainable params: 20,490 (80.04 KB)

Non-trainable params: 0 (0.00 B)

matches hand count 20,490? True

```
In [12]: U.set_seed(U.SEED)
with U.Timer() as t_keras:
    hk = keras_model.fit(
        Xtr_k, y_train, validation_data=(Xte_k, y_test),
        epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=2,
    )
print(f"\ntotal training time: {t_keras.seconds:.1f}s")
```

Epoch 1/5

157/157 – 1s – 9ms/step – accuracy: 0.8293 – loss: 0.5944 – val_accuracy: 0.9190 – val_loss: 0.2438

Epoch 2/5

157/157 – 1s – 4ms/step – accuracy: 0.9469 – loss: 0.1812 – val_accuracy: 0.9465 – val_loss: 0.1632

Epoch 3/5

157/157 – 1s – 4ms/step – accuracy: 0.9641 – loss: 0.1193 – val_accuracy: 0.9585 – val_loss: 0.1206

Epoch 4/5

157/157 – 1s – 4ms/step – accuracy: 0.9750 – loss: 0.0873 – val_accuracy: 0.9705 – val_loss: 0.0935

Epoch 5/5

157/157 – 1s – 4ms/step – accuracy: 0.9804 – loss: 0.0665 – val_accuracy: 0.9755 – val_loss: 0.0789

total training time: 4.1s

```
In [13]: pred_keras = keras_model.predict(Xte_k, batch_size=256,
      verbose=0).argmax(axis=1)
      m = U.evaluate(y_test, pred_keras, N_CLASSES)

      hist_keras = {"loss": [float(v) for v in hk.history["loss"]],
                    "val_acc": [float(v) for v in hk.history["val_accuracy"]]}

      results.append(U.RunResult(
          framework="TensorFlow/Keras", dataset="MNIST (10k subset)",
      model="CNN 2conv+fc",
          n_params=int(keras_model.count_params()), epochs=EPOCHS,
          train_seconds=round(t_keras.seconds, 2),
          train_loss=round(hist_keras["loss"][-1], 4),
          history=hist_keras, **m,
      ))
      print(f"accuracy {m['test_accuracy']:.4f}    macro-F1
            {m['f1_macro']:.4f}")
```

accuracy 0.9755 macro-F1 0.9756

5. Cách cài đặt C — PyTorch

Vấn kiến trúc ấy, dưới dạng một `nn.Module`, với chu trình huấn luyện viết ra tường minh. Nhánh này dùng GPU.

```
In [14]: U.set_seed(U.SEED)

class MnistCNN(nn.Module):
    def __init__(self, in_ch, n_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_ch, CH1, K, padding=PAD),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(CH1, CH2, K, padding=PAD),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(FLAT, n_classes),
        )

    def forward(self, x):
        x = self.features(x)          # feature extraction
        x = self.classifier(x)        # classification
        return x

torch_model = MnistCNN(C, N_CLASSES).to(DEVICE)
n_torch = sum(p.numel() for p in torch_model.parameters())

print(torch_model)
print(f"\nparameters: {n_torch:,}    matches hand count? {n_torch ==
EXPECTED_PARAMS}")
```

```
MnistCNN(
  (features): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=1568, out_features=10, bias=True)
  )
)

parameters: 20,490    matches hand count? True
```

```
In [15]: def train_torch(model, Xtr, ytr, Xte, yte, epochs=EPOCHS,
batch_size=BATCH_SIZE, lr=LR, log=True):
    # Explicit loop: zero grad -> forward -> loss -> backward -> update.
    loader = DataLoader(TensorDataset(torch.tensor(Xtr),
torch.tensor(ytr)),
                        batch_size=batch_size, shuffle=True)
    Xte_t = torch.tensor(Xte).to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    history = {"loss": [], "val_acc": []}

    with U.Timer() as t:
        for epoch in range(1, epochs + 1):
            model.train()
            running, nb = 0.0, 0
            for xb, yb in loader:
                xb, yb = xb.to(DEVICE), yb.to(DEVICE)

                optimizer.zero_grad()
                logits = model(xb)
                loss = criterion(logits, yb)
                loss.backward()
                optimizer.step()

                running += loss.item(); nb += 1

            model.eval()
            with torch.no_grad():
                preds = []
                for i in range(0, len(Xte_t), 1024):
                    preds.append(model(Xte_t[i:i +
1024]).argmax(1).cpu().numpy())
                acc = float((np.concatenate(preds) == yte).mean())

            history["loss"].append(running / nb)
            history["val_acc"].append(acc)
            if log:
                print(f"epoch {epoch}/{epochs}  loss {running/nb:.4f}
test_acc {acc:.4f}")

        return history, t.seconds

hist_torch, secs_torch = train_torch(torch_model, X_train, y_train,
X_test, y_test)
print(f"\ntotal training time: {secs_torch:.1f}s")
```

epoch 1/5 loss 0.5627 test_acc 0.9420

epoch 2/5 loss 0.1663 test_acc 0.9620

epoch 3/5 loss 0.1044 test_acc 0.9765

epoch 4/5 loss 0.0762 test_acc 0.9780

epoch 5/5 loss 0.0569 test_acc 0.9835

total training time: 1.9s

```
In [16]: torch_model.eval()
         with torch.no_grad():
             pred_torch =
             torch_model(torch.tensor(X_test).to(DEVICE)).argmax(1).cpu().numpy()
             m = U.evaluate(y_test, pred_torch, N_CLASSES)

         results.append(U.RunResult(
             framework="PyTorch", dataset="MNIST (10k subset)", model="CNN
             2conv+fc",
             n_params=n_torch, epochs=EPOCHS,
             train_seconds=round(secs_torch, 2),
             train_loss=round(hist_torch["loss"][-1], 4),
             history=hist_torch, **m,
         ))
         print(f"accuracy {m['test_accuracy']:.4f}   macro-F1
         {m['f1_macro']:.4f}")
```

accuracy 0.9835 macro-F1 0.9836

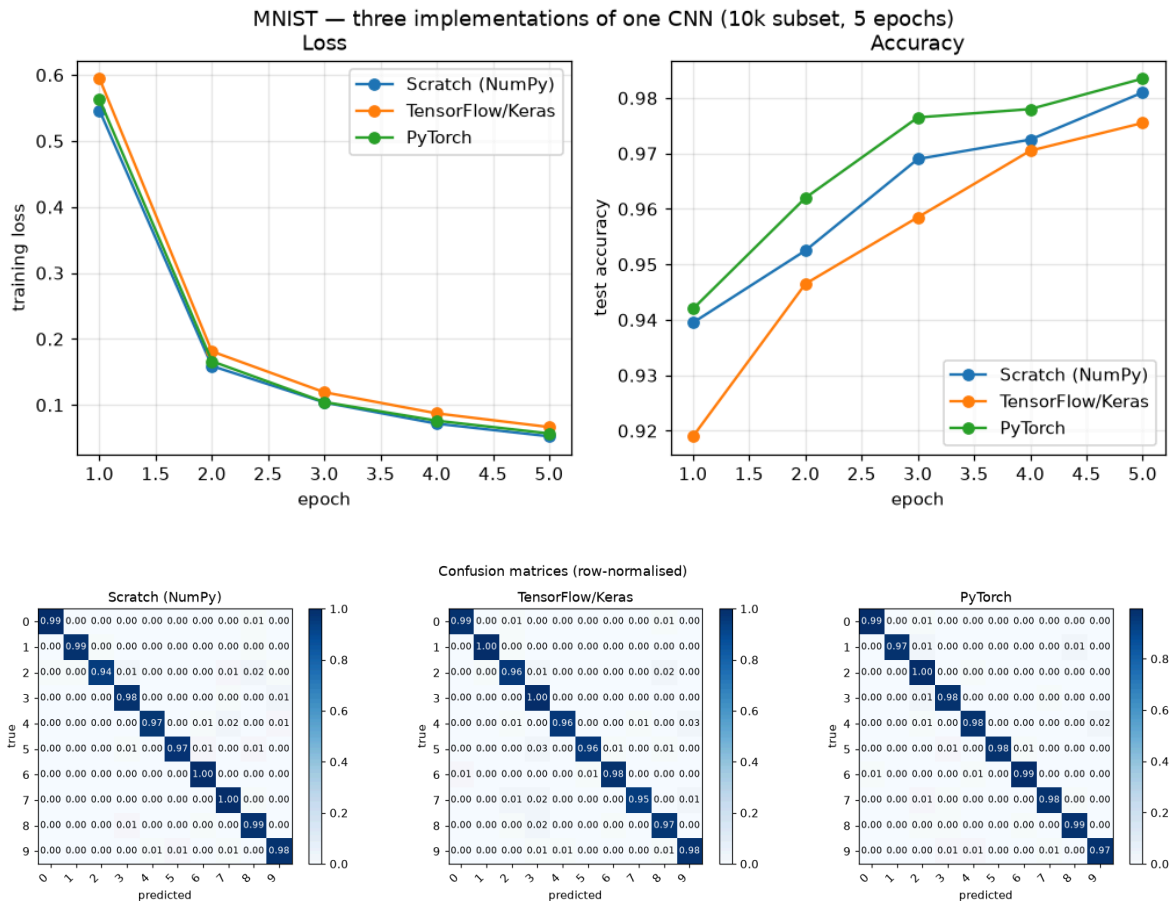
6. So sánh trên cùng một tập con

```
In [17]: table = U.results_table(results)
         display(table)
```

	frame work	dataset	mode l	n_p ara ms	ep oc hs	train _sec onds	trai n_lo ss	test_ accu racy	precisi on_ma cro	recall _mac ro	f1_ mac ro
0	Scratch (NumPy)	MNIST (10k subset)	CNN 2conv +fc	204 90	5	36.24	0.05 25	0.981 0	0.9813	0.981 0	0.98 10
1	Tensor Flow/K eras	MNIST (10k subset)	CNN 2conv +fc	204 90	5	4.09	0.06 65	0.975 5	0.9765	0.975 4	0.97 56
2	PyTorch	MNIST (10k subset)	CNN 2conv +fc	204 90	5	1.95	0.05 69	0.983 5	0.9838	0.983 5	0.98 36

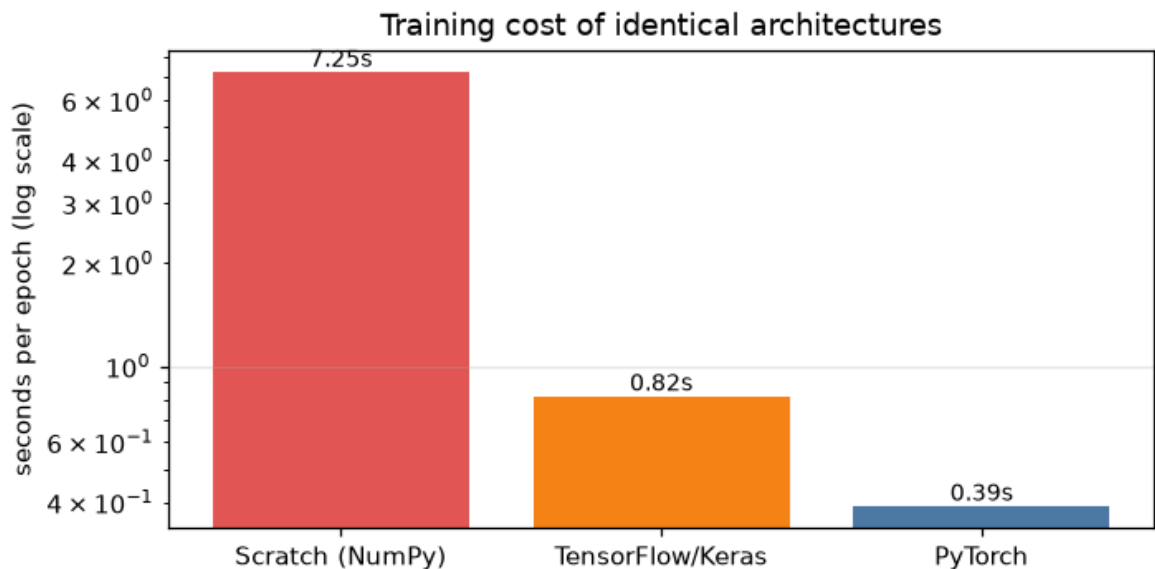
```
In [18]: U.plot_history({r.framework: r.history for r in results},
                        title="MNIST — three implementations of one CNN (10k
subset, 5 epochs)")
plt.show()

fig, axes = plt.subplots(1, 3, figsize=(15, 4.2))
for ax, r in zip(axes, results):
    U.plot_confusion(r.confusion, CLASSES, title=r.framework, ax=ax)
fig.suptitle("Confusion matrices (row-normalised)")
fig.tight_layout()
plt.show()
```



```
In [19]: # Training cost per epoch, on a log scale – the implementations differ
         # by orders of magnitude.
fig, ax = plt.subplots(figsize=(7, 3.4))
names = [r.framework for r in results]
secs = [r.train_seconds / r.epochs for r in results]
bars = ax.bar(names, secs, color=["#E45756", "#F58518", "#4C78A8"])
ax.set_yscale("log")
ax.set_ylabel("seconds per epoch (log scale)")
ax.set_title("Training cost of identical architectures")
for b, s in zip(bars, secs):
    ax.text(b.get_x() + b.get_width() / 2, s, f"{s:.2f}s", ha="center",
            va="bottom", fontsize=9)
ax.grid(alpha=0.3, axis="y")
plt.show()

slowest, fastest = max(secs), min(secs)
print(f"slowest / fastest = {slowest/fastest:.0f}x "
      "- same mathematics, different implementation and hardware")
```



slowest / fastest = 19x – same mathematics, different implementation and hardware

Soi các dự đoán sai (§44, bước 8 của tài liệu)

Mô hình nhầm những chữ số nào còn đáng đọc hơn con số độ chính xác. Dưới đây là các ảnh kiểm tra mà mô hình PyTorch đoán sai.

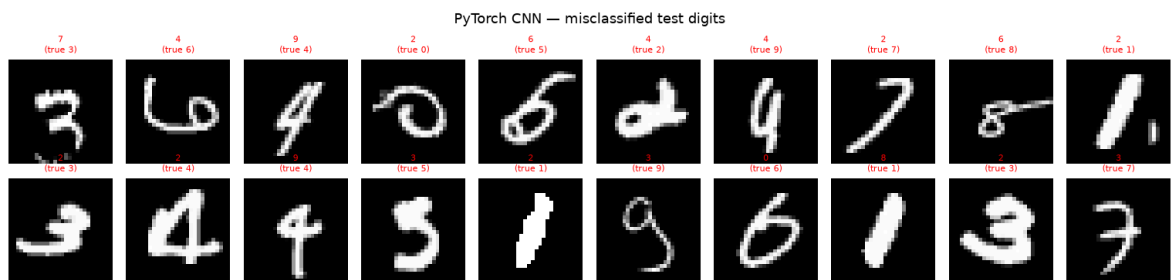
```
In [20]: wrong = np.where(pred_torch != y_test)[0]
print(f"{len(wrong)} wrong out of {len(y_test)} ({len(wrong)/len(y_test):.2%}")

if len(wrong):
    sel = wrong[:20]
    U.plot_samples(X_test[sel], y_test[sel], CLASSES, n=len(sel),
                  preds=pred_torch[sel],
                  title="PyTorch CNN - misclassified test digits")

    plt.show()

    cm = np.array(results[-1].confusion)
    np.fill_diagonal(cm, 0)
    pairs = np.vstack(np.unravel_index(np.argsort(cm.ravel())[:-1],
cm.shape))[0][:5]
    print("\nmist confused pairs (true -> predicted):")
    for i, j in pairs:
        if cm[i, j]:
            print(f" {CLASSES[i]} -> {CLASSES[j]}: {cm[i, j]}")
```

33 wrong out of 2000 (1.65%)



most confused pairs (true -> predicted):

```
1 -> 8: 3
1 -> 2: 3
3 -> 2: 3
4 -> 9: 3
9 -> 3: 2
```

7. Cái giá của việc dùng tập con

Bảng ba bên ở trên công bằng nhưng bị quan: 10.000 ảnh chỉ bằng một phần sáu MNIST. Chạy lại hai nhánh dùng framework trên **toàn bộ 60.000** ảnh cho thấy tập con đã đánh đổi bao nhiêu độ chính xác — và tách bạch "kiến trúc này yếu" với "lần chạy này ngắn".

```

In [21]: Xtr_f, ytr_f, Xte_f, yte_f, _ = U.load_mnist(verbose=True)

full_results = []

# --- PyTorch, full data ---
U.set_seed(U.SEED)
torch_full = MnistCNN(C, N_CLASSES).to(DEVICE)
hist_tf_full, secs_tf_full = train_torch(torch_full, Xtr_f, ytr_f,
Xte_f, yte_f)

torch_full.eval()
with torch.no_grad():
    preds = []
    Xte_ft = torch.tensor(Xte_f).to(DEVICE)
    for i in range(0, len(Xte_ft), 1024):
        preds.append(torch_full(Xte_ft[i:i +
1024]).argmax(1).cpu().numpy())
    pred_full_torch = np.concatenate(preds)

m = U.evaluate(yte_f, pred_full_torch, N_CLASSES)
full_results.append(U.RunResult(
    framework="PyTorch", dataset="MNIST (full 60k)", model="CNN
2conv+fc",
    n_params=n_torch, epochs=EPOCHS, train_seconds=round(secs_tf_full,
2),
    train_loss=round(hist_tf_full["loss"][-1], 4), history=hist_tf_full,
**m,
))
print(f"\nPyTorch full-data: accuracy {m['test_accuracy']:.4f} macro-F1
{m['f1_macro']:.4f}")

```

MNIST: train (60000, 1, 28, 28) / test (10000, 1, 28, 28), 10 classes

epoch 1/5 loss 0.1909 test_acc 0.9812

epoch 2/5 loss 0.0584 test_acc 0.9855

epoch 3/5 loss 0.0446 test_acc 0.9855

epoch 4/5 loss 0.0355 test_acc 0.9866

epoch 5/5 loss 0.0299 test_acc 0.9875

PyTorch full-data: accuracy 0.9875 macro-F1 0.9874

```

In [22]: # --- Keras, full data ---
U.set_seed(U.SEED)
keras_full = keras.Sequential([
    keras.layers.Input(shape=(H, W, C)),
    keras.layers.Conv2D(CH1, K, padding="same", activation="relu"),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(CH2, K, padding="same", activation="relu"),
    keras.layers.MaxPooling2D(2),
    keras.layers.Flatten(),
    keras.layers.Dense(N_CLASSES),
], name="mnist_cnn_full")
keras_full.compile(optimizer=keras.optimizers.Adam(learning_rate=LR),

loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
               metrics=["accuracy"])

with U.Timer() as t_kf:
    hkf = keras_full.fit(np.transpose(Xtr_f, (0, 2, 3, 1)), ytr_f,
                        validation_data=(np.transpose(Xte_f, (0, 2, 3,
1))), yte_f),
                        epochs=EPOCHS, batch_size=BATCH_SIZE,
verbose=2)

pred_full_keras = keras_full.predict(np.transpose(Xte_f, (0, 2, 3, 1)),
                                     batch_size=512,
verbose=0).argmax(axis=1)
m = U.evaluate(yte_f, pred_full_keras, N_CLASSES)
full_results.append(U.RunResult(
    framework="TensorFlow/Keras", dataset="MNIST (full 60k)", model="CNN
2conv+fc",
    n_params=int(keras_full.count_params()), epochs=EPOCHS,
    train_seconds=round(t_kf.seconds, 2),
    train_loss=round(float(hkf.history["loss"][-1]), 4),
    history={"loss": [float(v) for v in hkf.history["loss"]],
            "val_acc": [float(v) for v in
hkf.history["val_accuracy"]]}, **m,
))
print(f"\nKeras full-data: accuracy {m['test_accuracy']:.4f} macro-F1
{m['f1_macro']:.4f}")

```

Epoch 1/5

938/938 – 4s – 5ms/step – accuracy: 0.9412 – loss: 0.2020 – val_accuracy: 0.9768 – val_loss: 0.0693

Epoch 2/5

938/938 – 4s – 4ms/step – accuracy: 0.9815 – loss: 0.0607 – val_accuracy: 0.9831 – val_loss: 0.0526

Epoch 3/5

938/938 - 4s - 4ms/step - accuracy: 0.9866 - loss: 0.0438 - val_accuracy: 0.9838 - val_loss: 0.0509

Epoch 4/5

938/938 - 4s - 4ms/step - accuracy: 0.9899 - loss: 0.0343 - val_accuracy: 0.9846 - val_loss: 0.0494

Epoch 5/5

938/938 - 4s - 4ms/step - accuracy: 0.9920 - loss: 0.0275 - val_accuracy: 0.9855 - val_loss: 0.0468

Keras full-data: accuracy 0.9855 macro-F1 0.9854

```
In [23]: display(U.results_table(results + full_results))

sub_best = max(r.test_accuracy for r in results)
full_best = max(r.test_accuracy for r in full_results)
print(f"\nbest on 10k subset : {sub_best:.4f}")
print(f"best on full 60k : {full_best:.4f}")
print(f"cost of the subset : {full_best - sub_best:+.4f} accuracy")
```

	frame work	dataset	mode l	n_p ara ms	ep oc hs	train _sec onds	trai n_lo ss	test_ accu racy	precisi on_ma cro	recall _mac ro	f1_ mac ro
0	Scratch (NumPy)	MNIST (10k subset)	CNN 2conv +fc	204 90	5	36.24	0.05 25	0.981 0	0.9813	0.981 0	0.98 10
1	Tensor Flow/K eras	MNIST (10k subset)	CNN 2conv +fc	204 90	5	4.09	0.06 65	0.975 5	0.9765	0.975 4	0.97 56
2	PyTorch	MNIST (10k subset)	CNN 2conv +fc	204 90	5	1.95	0.05 69	0.983 5	0.9838	0.983 5	0.98 36
3	PyTorch	MNIST (full 60k)	CNN 2conv +fc	204 90	5	9.06	0.02 99	0.987 5	0.9875	0.987 5	0.98 74
4	Tensor Flow/K eras	MNIST (full 60k)	CNN 2conv +fc	204 90	5	19.19	0.02 75	0.985 5	0.9855	0.985 4	0.98 54

```
best on 10k subset : 0.9835
best on full 60k : 0.9875
cost of the subset : +0.0040 accuracy
```

8. Mỗi thành phần nằm ở đâu

```
In [24]: component_table = pd.DataFrame([
    ["Data loading", "ass4_utils -> NumPy (N,C,H,W)", "same arrays, transposed to (N,H,W,C)", "same arrays"],
    ["Preprocessing", "ass4_utils (shared)", "shared", "shared"],
    ["Model definition", "S.Sequential([...])", "keras.Sequential([...])", "nn.Module"],
    ["Convolution", "S.Conv2D (im2col + matmul)", "keras.layers.Conv2D", "nn.Conv2d"],
    ["Activation", "S.ReLU (mask multiply)", "activation='relu'", "nn.ReLU"],
    ["Pooling", "S.MaxPool2D (argmax routing)", "keras.layers.MaxPooling2D", "nn.MaxPool2d"],
    ["Flatten", "S.Flatten (reshape)", "keras.layers.Flatten", "nn.Flatten"],
    ["Dense layer", "S.Dense (x@W+b)", "keras.layers.Dense", "nn.Linear"],
    ["Loss", "S.SoftmaxCrossEntropy", "SparseCategoricalCrossentropy", "nn.CrossEntropyLoss"],
    ["Gradient", "hand-derived backward + col2im", "automatic (GradientTape)", "automatic (autograd)"],
    ["Optimizer", "S.Adam (written out)", "keras.optimizers.Adam", "torch.optim.Adam"],
    ["Training loop", "explicit for-loop", "model.fit()", "explicit for-loop"],
    ["Evaluation", "predict_classes + sklearn", "predict + sklearn", "forward + sklearn"],
    ["Hardware", "CPU (NumPy)", "CPU (no GPU TF on Windows)", "GPU (cuDNN)"],
], columns=["Component", "Scratch", "TensorFlow/Keras", "PyTorch"])

display(component_table)
```

	Component	Scratch	TensorFlow/Keras	PyTorch
0	Data loading	ass4_utils -> NumPy (N,C,H,W)	same arrays, transposed to (N,H,W,C)	same arrays
1	Preprocessing	ass4_utils (shared)	shared	shared
2	Model definition	S.Sequential([...])	keras.Sequential([...])	nn.Module
3	Convolution	S.Conv2D (im2col + matmul)	keras.layers.Conv2D	nn.Conv2d
4	Activation	S.ReLU (mask multiply)	activation='relu'	nn.ReLU
5	Pooling	S.MaxPool2D (argmax routing)	keras.layers.MaxPooling2D	nn.MaxPool2d
6	Flatten	S.Flatten (reshape)	keras.layers.Flatten	nn.Flatten
7	Dense layer	S.Dense (x@W+b)	keras.layers.Dense	nn.Linear
8	Loss	S.SoftmaxCrossEntropy	SparseCategoricalCrossentropy	nn.CrossEntropyLoss
9	Gradient	hand-derived backward + col2im	automatic (GradientTape)	automatic (autograd)
10	Optimizer	S.Adam (written out)	keras.optimizers.Adam	torch.optim.Adam
11	Training loop	explicit for-loop	model.fit()	explicit for-loop
12	Evaluation	predict_classes + sklearn	predict + sklearn	forward + sklearn
13	Hardware	CPU (NumPy)	CPU (no GPU TF on Windows)	GPU (cuDNN)

```
In [25]: os.makedirs("results", exist_ok=True)
with open("results/02_mnist.json", "w") as f:
    json.dump({"meta": {k: v for k, v in meta.items()},
              "results": [r.__dict__ for r in results],
              "full_results": [r.__dict__ for r in full_results],
              "component_table":
component_table.to_dict(orient="records")},
              f, indent=2, default=str)
print("saved results/02_mnist.json")
```

saved results/02_mnist.json

Lưu lại mô hình đã huấn luyện

Tập JSON ở trên ghi lại các mô hình **đạt** bao nhiêu điểm. Nó không ghi lại các mô hình **là** gì — muốn có lại những con số này thì phải huấn luyện lại từ đầu.

`U.save_model` ghi từng nhánh theo định dạng gốc của framework tương ứng, vào `results/models/<notebook>/`, kèm một tập JSON đi cùng ghi rõ kiến trúc và điểm số mà đúng bộ trọng số đó đạt được:

nhánh	tập	tự nạp lại được?
Scratch (NumPy)	<code>.npz</code> , mỗi tham số của mỗi tầng là một mảng	không
TensorFlow/Keras	<code>.keras</code> , đồ thị + trọng số	được
PyTorch	<code>.pt</code> , <code>state_dict</code>	không

`.pt` và `.npz` chỉ chứa trọng số, nên nạp lại chúng đồng nghĩa với việc phải dựng lại kiến trúc trước rồi truyền đối tượng mới vào `U.load_model(..., model=...)`. Đây đúng là giao kèo mà `variants.py` dùng cho `results/variant_cache/`.

```

In [26]: # Weights are the model; metrics only describe it. Persist all three
         legs.
         TAG = "02_mnist"

         by_fw = {r.framework: r for r in results}

         for obj, fw, stem in [
             (scratch_model, "Scratch (NumPy)", "cnn_scratch"),
             (keras_model, "TensorFlow/Keras", "cnn_keras"),
             (torch_model, "PyTorch", "cnn_torch"),
         ]:
             r = by_fw[fw]
             U.save_model(obj, f"{stem}_subset", TAG, architecture=r.model,
                           dataset=r.dataset,
                           epochs=r.epochs, test_accuracy=r.test_accuracy,
                           f1_macro=r.f1_macro)

         # The full-data reruns – scratch NumPy is not rerun at this size, so two
         legs.
         by_fw_full = {r.framework: r for r in full_results}

         for obj, fw, stem in [
             (keras_full, "TensorFlow/Keras", "cnn_keras"),
             (torch_full, "PyTorch", "cnn_torch"),
         ]:
             r = by_fw_full[fw]
             U.save_model(obj, f"{stem}_full", TAG, architecture=r.model,
                           dataset=r.dataset,
                           epochs=r.epochs, test_accuracy=r.test_accuracy,
                           f1_macro=r.f1_macro)

         display(U.list_models(TAG)[["name", "framework", "n_params", "dataset",
                                     "test_accuracy"]])

```

```

saved results\models\02_mnist\cnn_scratch_subset.npz (20,490 params, 76
KB)
saved results\models\02_mnist\cnn_keras_subset.keras (20,490 params, 272
KB)
saved results\models\02_mnist\cnn_torch_subset.pt (20,490 params, 83 KB)
saved results\models\02_mnist\cnn_keras_full.keras (20,490 params, 272
KB)
saved results\models\02_mnist\cnn_torch_full.pt (20,490 params, 83 KB)

```

	name	framework	n_params	dataset	test_accuracy
0	cnn_keras_full	keras	20490	MNIST (full 60k)	0.9855
1	cnn_keras_subset	keras	20490	MNIST (10k subset)	0.9755
2	cnn_scratch_subset	scratch	20490	MNIST (10k subset)	0.9810
3	cnn_torch_full	torch	20490	MNIST (full 60k)	0.9875
4	cnn_torch_subset	torch	20490	MNIST (10k subset)	0.9835

9. Bộ dữ liệu này cho thấy điều gì

Ba cách cài đặt lại thống nhất với nhau, và lần này kiến trúc là CNN. Vẫn 20.490 tham số, vẫn đúng những kích thước ấy ở mọi giai đoạn, độ chính xác nằm trong dải nhiều của nhau. Tích chập `im2col` viết tay và cuDNN tính ra cùng một hàm số.

Khoảng cách chi phí thì khổng lồ, và nó không nằm ở phần toán. Nhánh scratch chậm hơn nhánh GPU vài bậc độ lớn trên mỗi epoch. Kiến trúc không đổi chút nào — chỉ đổi ai thực hiện các phép nhân ma trận và trên phần cứng nào. Đây là minh hoạ rõ nhất cho luận điểm của bài giảng: trừu tượng hoá thay đổi **mức độ nhìn thấy**, không thay đổi phép tính bên dưới.

Ở đây tích chập xứng đáng có mặt theo cách mà ở notebook 1 nó không thể. Hai tầng tích chập giữ 4.800 tham số và làm phần trích xuất đặc trưng; một tầng kết nối đầy đủ giữ 15.690 — 77 % mô hình — mà chỉ làm việc đọc ra câu trả lời. Một MLP chạy thẳng trên điểm ảnh thô sẽ phải trả cái giá kết nối đầy đủ ấy ở **mọi** tầng.

MNIST gần như đã bão hoà. Ngay cả năm epoch trên một phần sáu dữ liệu cũng cho kết quả cao, và các lỗi còn lại đúng là những chữ số mập mờ thật — nhầm 4/9 và 3/5 nhìn thấy được trong ma trận nhầm lẫn ở trên. Đó là lý do phải cẩn thận: MNIST không phân biệt được kiến trúc tốt với kiến trúc tạm đủ.

Và đó đúng là lý do notebook tiếp theo dùng CIFAR-10: ảnh màu, kết cấu đời thực, và là bộ dữ liệu mà chính mạng CNN nhỏ này sẽ chật vật thấy rõ — làm cho những cải tiến kiến trúc ở `04_compare.ipynb` đáng đo đạc.